

Technical Report

Informed Search and Adversarial Search in Artificial Intelligence

Dry-Run Analysis of A* Search and Minimax with Alpha-Beta Pruning

Algorithms covered:

A* Search (Informed Graph Search) · Minimax Algorithm · Alpha-Beta Pruning

Abstract

This report presents a detailed dry-run analysis of two classical Artificial Intelligence search techniques applied to representative problems: informed graph search using the **A* algorithm** on a weighted, heuristic-annotated graph, and adversarial two-player decision-making using the **Minimax algorithm enhanced with Alpha-Beta pruning** on a depth-2 game tree. For the A* scenario, the algorithm is traced iteration-by-iteration — recording the open list, closed list, and $g(n)$, $h(n)$, $f(n)$ values at each step — to identify the optimal path from a start node to a goal node and to verify that the path returned is cost-optimal. For the Minimax scenario, the game tree is evaluated left-to-right with running alpha and beta bounds recorded at each node, identifying the best move for the maximizing player, the final minimax value, and any branches pruned as a result of the alpha-beta cutoff condition. The results confirm the theoretical guarantees of both algorithms: A* returns the lowest-cost path when its heuristic is admissible, and Alpha-Beta pruning reaches the exact same decision as unpruned Minimax while skipping branches that cannot affect the outcome.

Table of Contents

- 1. Introduction
- 2. Scenario 1 — Informed Search: A* Algorithm Dry Run
- 3. Scenario 2 — Adversarial Search: Minimax with Alpha-Beta Pruning Dry Run
- 4. Conclusion

1. Introduction

Search is the foundational problem-solving mechanism in Artificial Intelligence: an agent must find a sequence of actions or decisions that leads from an initial state to a goal state, or to the best possible outcome, within a defined environment. This report examines two problems that sit at opposite ends of the AI search spectrum. The first is a single-agent, cost-driven pathfinding problem, solved using **A* Search**, an informed search algorithm that combines the actual cost incurred so far, $g(n)$, with an estimate of the remaining cost, $h(n)$, to guarantee an optimal solution whenever the heuristic is admissible. The second is a two-agent, zero-sum, adversarial decision problem, solved using the **Minimax algorithm**, which models an opponent that always acts to minimize the outcome for the maximizing player, and refined with **Alpha-Beta pruning**, which eliminates provably irrelevant branches of the search tree without altering the final decision.

For each scenario, the report follows a consistent structure: a statement of the problem as given, the objective of the analysis, an explanation of the algorithm applied and why it fits the problem, and a full step-by-step dry run with a results and analysis discussion interpreting the outcome.

2. Scenario 1 — Informed Search: A* Algorithm Dry Run

2.1 Problem Statement

A weighted, undirected graph of seven relevant nodes (A, B, C, D, E, G, and an unused node H) models a set of locations connected by paths of varying cost. Each node is annotated with a heuristic value $h(n)$, representing an estimate of the remaining cost to the goal node G. The task is to find the lowest-cost path from the start node A to the goal node G.

2.2 Objective

- Perform a complete dry run of the A* Search algorithm from A to G.
- At each iteration, record the current node, the open list, the closed list, and the $g(n)$, $h(n)$, and $f(n)$ values.
- Identify the optimal path returned by the algorithm and verify its total cost.

2.3 Algorithm / Theoretical Approach

A* Search maintains an open list of frontier nodes ordered by $f(n) = g(n) + h(n)$, where $g(n)$ is the exact cost of the path from the start node to n , and $h(n)$ is a heuristic estimate of the remaining cost from n to the goal. At each step, the node with the lowest $f(n)$ is removed from the open list, moved to the closed list, and expanded; any neighbour reached via a cheaper path than previously recorded has its g , f values updated. Because the heuristic values supplied never overestimate the true remaining cost on this graph (admissibility), A* is guaranteed to terminate with an optimal, lowest-cost path the first time the goal node is selected for expansion.

Graph edges (weights) used: A-B = 2, A-C = 4, B-C = 3, B-D = 7, B-E = 2, C-E = 3, D-E = 2, D-G = 2
Heuristic values $h(n)$: A = 7, B = 6, C = 4, D = 3, E = 2, G = 0

2.4 Dry Run

Iter	Current Node	$g(n)$	$h(n)$	$f(n)=g+h$	Open List (node: f)	Closed List
0	—	—	—	—	A:7	—
1	A	0	7	7	B:8, C:8	A
2	B	2	6	8	C:8, D:12, E:6	A, B
3	E	4	2	6	C:8, D:9	A, B, E
4	C	4	4	8	D:9	A, B, E, C
5	D	6	3	9	G:8	A, B, E, C, D
6	G (Goal!)	8	0	8	—	A, B, E, C, D, G

Explanation of key steps:

- Iteration 1:** Expanding A generates B ($g=2$, $h=6$, $f=8$) and C ($g=4$, $h=4$, $f=8$).
- Iteration 2:** B and C tie at $f=8$; B is expanded first. The route to C via B ($f=9$) is worse than the existing entry and is discarded. D ($f=12$) and E ($f=6$) are generated.
- Iteration 3:** E has the lowest f (6). Expanding E finds a cheaper route to D ($g=6$, $f=9$), replacing the earlier, more expensive entry; the route to C via E ($f=11$) is worse than the existing entry and is discarded.
- Iteration 4:** C ($f=8$) is expanded next; all of its neighbours (A, B, E) are already closed, so no new nodes are added.
- Iteration 5:** D ($f=9$) is expanded, generating G with $g=8$, $h=0$, $f=8$.
- Iteration 6:** G, the only remaining open node, is selected and is the goal — the search terminates.

2.5 Results & Analysis

The algorithm terminates with the goal node G reached at a cost of $g(G) = 8$. Tracing parent pointers back from G gives the path $A \rightarrow B \rightarrow E \rightarrow D \rightarrow G$, with individual edge costs of 2, 2, 2, and 2 respectively. No cheaper alternative route exists in the open or closed lists at termination, which confirms that the returned path is optimal — consistent with the theoretical guarantee that A* returns an optimal solution whenever its heuristic is admissible.

1. Optimal Path: $A \rightarrow B \rightarrow E \rightarrow D \rightarrow G$ (2 + 2 + 2 + 2)

2. Total Path Cost: 8

3. Scenario 2 — Adversarial Search: Minimax with Alpha-Beta Pruning Dry Run

3.1 Problem Statement

A two-ply, zero-sum game tree is rooted at a MAX node with two MIN children. The left MIN node has three leaf children with values 3, 5, and 6; the right MIN node has three leaf children with values 9, 1, and 2. The task is to determine the value MAX can guarantee and the branches that can be safely skipped without affecting that decision.

3.2 Objective

- Perform a full dry run of the Minimax algorithm with Alpha-Beta pruning, evaluating the tree strictly left to right.
- At each node, record the running alpha (α) and beta (β) bounds, the value selected, and any nodes pruned.
- Determine the best move for MAX, the final minimax value, and list all pruned nodes.

3.3 Algorithm / Theoretical Approach

Minimax assigns the maximizing player (MAX) the move that yields the highest achievable value, assuming the minimizing player (MIN) always replies with the move that yields the lowest value for MAX. Alpha-Beta pruning augments this traversal with two running bounds: alpha, the best value MAX can already guarantee along the path so far, and beta, the best value MIN can already guarantee. Whenever alpha becomes greater than or equal to beta at a node, the remaining, unexplored children of that node are skipped (pruned), because the already-explored alternative is provably at least as good regardless of what the pruned branch would return.

Tree structure:
MAX (root)
├─ MIN1 → leaves: 3, 5, 6
└─ MIN2 → leaves: 9, 1, 2

3.4 Dry Run

Node Visited	α (before → after)	β (before → after)	Value / Action
Leaf 3 (under MIN1)	$-\infty$	$\infty \rightarrow 3$	$\beta = \min(\infty, 3) = 3$
Leaf 5 (under MIN1)	$-\infty$	$3 \rightarrow 3$	$\beta = \min(3, 5) = 3$ (no change)
Leaf 6 (under MIN1)	$-\infty$	$3 \rightarrow 3$	$\beta = \min(3, 6) = 3$ (no change)
MIN1 result	$-\infty$	3	MIN1 = min(3, 5, 6) = 3
MAX (after MIN1)	$-\infty \rightarrow 3$	∞	$\alpha = \max(-\infty, 3) = 3$
Leaf 9 (under MIN2)	3	$\infty \rightarrow 9$	$\beta = \min(\infty, 9) = 9$; $\alpha(3) < \beta(9)$, continue
Leaf 1 (under MIN2)	3	$9 \rightarrow 1$	$\beta = \min(9, 1) = 1$; $\alpha(3) \geq \beta(1) \rightarrow$ cutoff
Leaf 2 (under MIN2)	—	—	PRUNED - never visited
MIN2 result	3	1	MIN2 = 1
MAX (root) result	3	∞	MAX = max(3, 1) = 3

Why leaf "2" is pruned: After leaf 1 is seen under MIN2, beta drops to 1. Since alpha = 3 (the value MAX can already guarantee via MIN1) is greater than or equal to beta = 1, MAX would never allow play to continue into this branch — MIN2 can only return a value of 1 or lower, which is strictly worse for MAX than the 3 already secured. The third child of MIN2 (value 2) is therefore never evaluated.

3.5 Results & Analysis

Both branches of the root are explored, but the third leaf of MIN2 is pruned as soon as the alpha-beta cutoff condition is met, illustrating that pruning removes only branches that cannot change the final decision. The value MAX would have obtained from a full, unpruned Minimax search (MIN2 = min(9, 1, 2) = 1) is identical to the value obtained with pruning (MIN2 = 1), confirming that Alpha-Beta pruning preserves decision quality exactly while reducing the number of nodes evaluated.

- 1. Best Move for MAX:** Choose the left branch (MIN1), which yields value 3.
- 2. Final Minimax Value:** 3
- 3. Pruned Nodes:** The leaf node with value 2 (third/rightmost child of MIN2)

4. Conclusion

Both dry runs confirm the theoretical guarantees underlying their respective algorithms. In Scenario 1, A* Search, guided by an admissible heuristic, expanded nodes in strict order of $f(n)$ and returned the path $A \rightarrow B \rightarrow E \rightarrow D \rightarrow G$ at the provably minimal cost of 8, correctly avoiding the higher-cost route through D directly from B. In Scenario 2, Alpha-Beta pruning reached the exact same minimax value (3) and the same best move for MAX as an unpruned Minimax search would have, while skipping the evaluation of one leaf node (value 2) once the alpha-beta cutoff condition ($\alpha \geq \beta$) was met at the MIN2 node.

Together, these results reinforce a central principle in AI search: optimality and efficiency are not opposing goals when the right algorithmic guarantees hold — A* achieves optimal-cost pathfinding without exhaustive search when its heuristic is admissible, and Alpha-Beta pruning achieves identical adversarial decisions to brute-force Minimax while exploring a strictly smaller portion of the search tree.